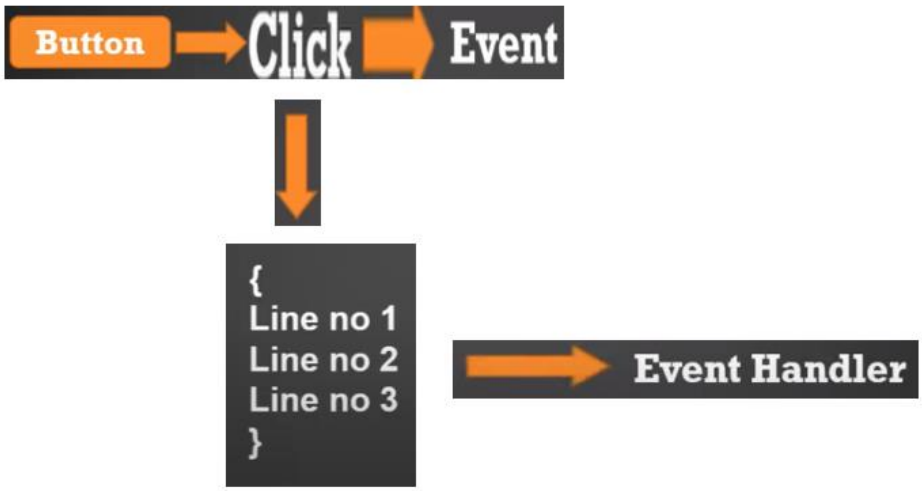
Event Handling in Java

What is an Event?



Change in the state of an object is known as “Event”.

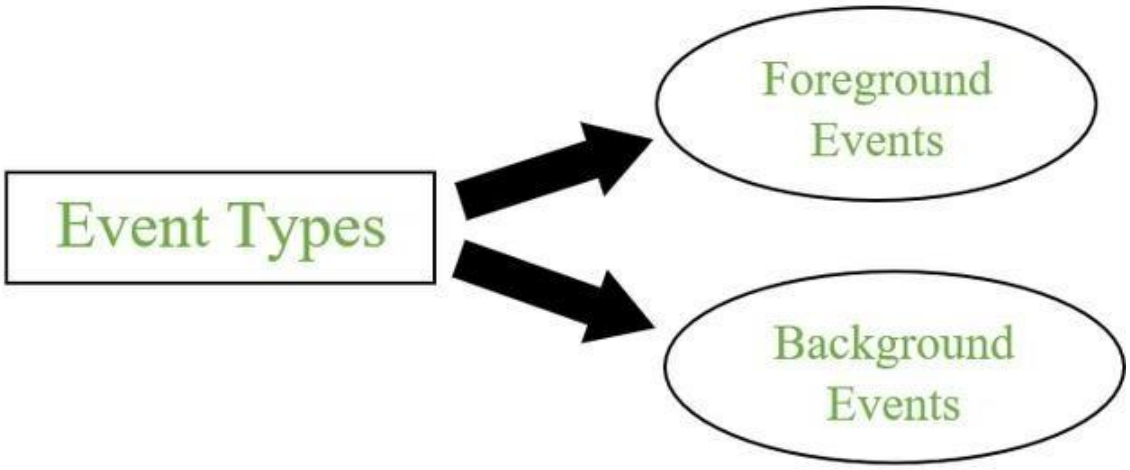
An **event** can be defined as changing the state of an object or behavior by performing actions. Actions can be a button click, cursor movement, keypress through keyboard or page scrolling, etc.
The **java.awt.event** package can be used to provide various event classes.

Example of Events:

- Clicking on a Button.
- Moving the Mouse.
- Entering a character through Keyboard.
- Selecting an item from the list.
- Scrolling the page, etc.

Classification of Events

- Foreground Events
- Background Events



Types of Events

1. Foreground Events

Foreground events are the events that require user interaction to generate, i.e., foreground events are generated due to interaction by the user on components in Graphic User Interface (**GUI**). Interactions are nothing but clicking on a button, scrolling the scroll bar, cursor moments, etc.

2. Background Events

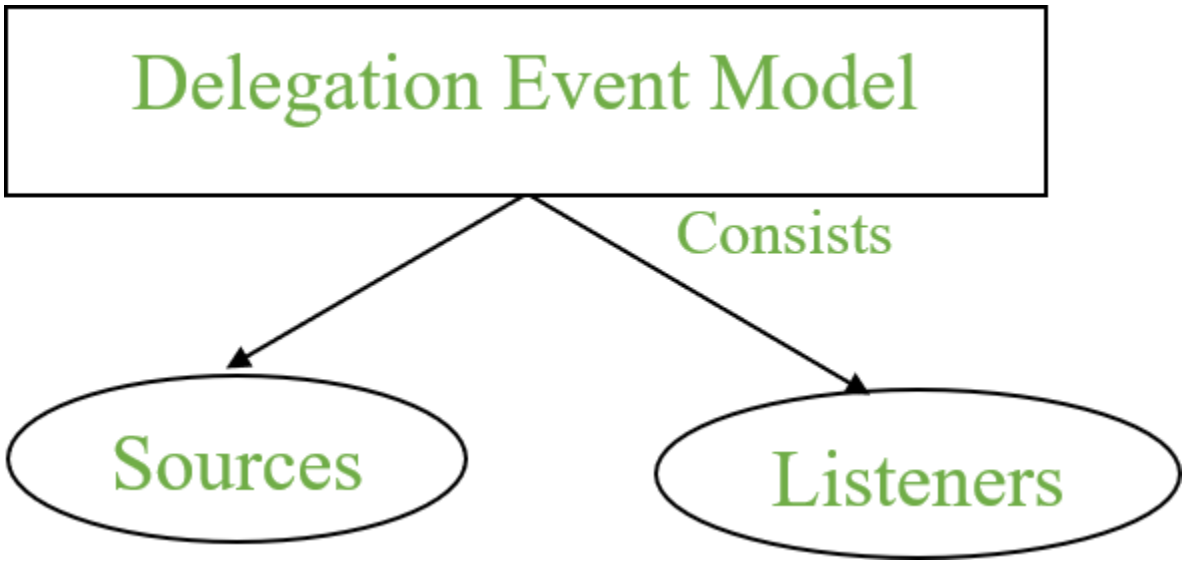
Events that don't require interactions of users to generate are known as background events. Examples of these events are operating system failures/interrupts, operation completion, etc.

Event Handling

It is a mechanism to **control the events** and to **decide what should happen after an event** occur. To handle the events, Java follows the ***Delegation Event model.***

Delegation Event model

- It has Sources and Listeners.



Delegation Event Model

- **Source:** Events are generated from the source. There are various sources like buttons, checkboxes, list, menu-item, choice, scrollbar, text components, windows, etc., to generate events.
- **Listeners:** Listeners are used for handling the events generated from the source. Each of these listeners represents interfaces that are responsible for handling events.

To perform Event Handling, we need to register the source with the listener.

Advantages of Event Handling:

- The benefit of this approach is that the user interface logic is completely separated from the logic that generates the event.
- The user interface element is able to delegate the processing of an event to the separate piece of code.
- In this model, Listener needs to be registered with the source object so that the listener can receive the event notification.
- This is an efficient way of handling the event because the event notifications are sent only to those listeners that want to receive them.

Registering the Source with Listener

Different Classes provide different registration methods.

Syntax:
`addTypeListener()`

where Type represents the type of event.

Example 1: For **KeyEvent** we use *addKeyListener()* to register.
Example 2:that For **ActionEvent** we use *addActionListener()* to register.

Event Classes in Java

Event Class	Listener Interface	Description
ActionEvent	ActionListener	An event that indicates that a component-defined action occurred like a button click or selecting an item from the menu-item list.
AdjustmentEvent	AdjustmentListener	The adjustment event is emitted by an Adjustable object like Scrollbar.
ComponentEvent	ComponentListener	An event that indicates that a component moved, the size changed or changed its visibility.
ContainerEvent	ContainerListener	When a component is added to a container (or) removed from it, then this event is generated by a container object.
FocusEvent	FocusListener	These are focus-related events, which include focus, focusin, focusout, and blur.
ItemEvent	ItemListener	An event that indicates whether an item was selected or not.

KeyEvent	KeyListener	An event that occurs due to a sequence of keypresses on the keyboard.
MouseEvent	MouseListener & MouseMotionListener	The events that occur due to the user interaction with the mouse (Pointing Device).

Event Class	Listener Interface	Description
MouseWheelEvent	MouseWheelListener	An event that specifies that the mouse wheel was rotated in a component.
TextEvent	TextListener	An event that occurs when an object's text changes.
WindowEvent	WindowListener	An event which indicates whether a window has changed its status or not.

Note: *As Interfaces contains abstract methods which need to implemented by the registered class to handle events.*

Different interfaces consists of different methods which are specified below.

Listener Interface	Methods
ActionListener	• actionPerformed()
AdjustmentListener	• adjustmentValueChanged()
ComponentListener	• componentResized() • componentShown() • componentMoved() • componentHidden()
ContainerListener	• componentAdded() • componentRemoved()
FocusListener	• focusGained() • focusLost()
ItemListener	• itemStateChanged()

Listener Interface	Methods
KeyListener	• keyTyped() • keyPressed() • keyReleased()
MouseListener	• mousePressed() • mouseClicked() • mouseEntered() • mouseExited() • mouseReleased()
MouseMotionListener	• mouseMoved() • mouseDragged()
MouseWheelListener	• mouseWheelMoved()
TextListener	• textChanged()
WindowListener	• windowActivated() • windowDeactivated() • windowOpened() • windowClosed() • windowClosing() • windowIconified() • windowDeiconified()

Flow of Event Handling

1. User Interaction with a component is required to generate an event.
2. The object of the respective event class is created automatically after event generation, and it holds all information of the event source.
3. The newly created object is passed to the methods of the registered listener.
4. The method executes and returns the result.

Code-Approaches

The three approaches for performing event handling are by placing the event handling code in one of the below-specified places.

1. Within Class
2. Other Class
3. Anonymous Class